

ネットワーク及び演習

第 4 回 echoClient.py と echoServer.py を読む

佐野 彰

本日の到達目標

- バイトとは何か、`b'Hello, world'` が何を表すかを説明できる
- ネットワークを流れるのはバイト列だけであることを説明し、`encode()` / `decode()` を使い分けられる
- `with` 文が何を自動化しているか説明できる
- `echoClient.py` と `echoServer.py` を、1 行ずつ意味を説明しながら読める
- サーバ側にソケットが 2 つある理由を説明できる

前回は通信の流れを見た。今回は、動いているコードのどの部分がある流れにあたるのかを対応づける。

1. バイトと文字列

バイト (byte) とは

- コンピュータがデータを扱う基本の単位。1 バイト = 8 ビット = 0~255 の整数 1 個
 - 16 進数で書くと 00 ~ ff の 2 桁
- ファイルの大きさ (KB, MB) も、通信量も、バイトの数で測る
- 文字も画像も音声も、最後はバイトの並び (バイト列) として保存・送信される
- 文字は、文字ごとに決めた番号で表す。英数字の番号は ASCII で決まっている

文字	H	e	l	l	o
番号 (10 進)	72	101	108	108	111
番号 (16 進)	48	65	6c	6c	6f

Hello という文字列は、5 バイトのバイト列 72 101 108 108 111 として保存される。

Python の bytes 型と b'Hello, world'

```
>>> data = b'Hello, world'
>>> type(data)
<class 'bytes'>
>>> len(data)
12
>>> data[0]
72
>>> list(b'Hello')
[72, 101, 108, 108, 111]
```

- `bytes` は数値 (0~255) の並びを表す型
- `b'...'` は bytes の書き方。 `b'Hello'` は、`H` `e` `l` `l` `o` の番号を並べたもの
- `data[0]` は文字 `'H'` ではなく、番号の `72`
- `b'...'` の中に直接書けるのは ASCII の文字だけ。 `b'あ'` は `SyntaxError`
- ASCII 以外のバイトは `\xe3` のように 16 進で書く
- 作った `bytes` は変更できない (`data[0] = 1` は `TypeError`)

ネットワークを流れるのはバイト列だけ



送れるのはバイト列だけ。encode と decode は、送る直前と受け取った直後の 2 か所にしかない。

- ケーブルや電波を流れるのは、バイトの並びだけ。「これは文字」という情報は一緒に運ばれない
- そのため、`sendall()` に渡せるのは `bytes` だけ。 `s.sendall('Hello')` は `TypeError` になる
- 整数も画像も、最後はバイト列にして送る。元のデータに戻す方法は、受け取る側のプログラムが知っている必要がある

str と bytes — 型と使い方

	str	bytes
中身	文字の並び	数値（0～255）の並び
書き方	'Hello'	b'Hello'
len()	文字の数	バイトの数
変換	.encode() で bytes になる	.decode() で str になる
使う場面	print() での表示、文字列の処理	sendall() / recv()、バイナリファイル

- 送る直前に encode()、受け取った直後に decode()。この 2 か所だけで変換する
- どんなバイト列にするかは文字コードで決まる。既定は UTF-8（encode('utf-8') と明示してもよい）

str と bytes の変換

```
'Hello, world'      # str
b'Hello, world'     # bytes

'あ'.encode()       # b'\xe3\x81\x82'
b'\xe3\x81\x82'.decode() # 'あ'

len('あ')           # 1 (文字の数)
len('あ'.encode())  # 3 (バイトの数)
```

- ASCII の文字は、`'A'` も `b'A'` も 1 バイトなので、見た目が似ていて紛らわしい
- 日本語を含めると差が出て、`len()` の値が一致しなくなる
- 送信側と受信側で、同じ文字コードを使う必要がある

```
s.sendall('こんにちは'.encode()) # 送る直前に encode
data = s.recv(1024)
print(data.decode())             # 受け取った直後に decode
```

文字コードが食い違うとどうなるか

```
t = 'こんにちは'
t.encode('utf-8')          # b'\xe3\x81\x93\xe3\x82\x93\xe3\x81\xab\xe3\x81\xa1\xe3\x81\xaf' (15 バイト)
t.encode('shift_jis')      # b'\x82\xb1\x82\xf1\x82\xc9\x82xbf\x82xcd' (10 バイト)
```

送る側 → 受ける側	結果
UTF-8 で encode → UTF-8 で decode	こんにちは (正しく戻る)
UTF-8 で encode → Shift_JIS で decode	繒雍 s 繒繒繒。繒ッ (文字化け。エラーにならない)
Shift_JIS で encode → UTF-8 で decode	UnicodeDecodeError (invalid start byte)
'こんにちは'.encode('ascii')	UnicodeEncodeError (ASCII に無い文字)

- 同じバイト列でも、どの文字コードで読むかで別の文字になる。バイト列には文字コードの情報が入っていない
- 文字化けはエラーにならないので、気づきにくい。repr() でバイト列を見ると原因を探しやすい
- echoServer.py は中身を解釈しないので、どの文字コードのバイト列でもそのまま返せる。食い違いが起きるのは、decode() する側

送る側と受ける側で、文字コードを合わせる。この科目では、既定の UTF-8 を使う。

repr() — 見ているデータの型を確かめる

```
>>> data = b'Hello'
>>> print(data)          # b'Hello'      ← Python 3 では bytes とわかる
>>> print(data.decode()) # Hello         ← str になった
>>> print(repr(data.decode())) # 'Hello'   ← クォート付きで str と明示される

>>> import datetime
>>> today = datetime.date.today()
>>> print(today)          # 2026-10-09    ← 文字列と見分けがつかない
>>> print(repr(today))    # datetime.date(2026, 10, 9)
```

- `print()` は人に見せる形 (`str()`)、`repr()` は元に戻せる形で表示する
- 通信のデバッグでは、空白・改行・文字コードの違いが見えないと原因を探せない
 - `print(repr(data))` なら、`b'Hello\r\n'` のように改行まで表示される

repr() で文字コードの食い違いを見つける

受け取った `data` を `decode()` したら、文字化けした。原因はどこか。

```
>>> data = 'こんにちは'.encode('utf-8')      # 相手が送ってきたバイト列とする
>>> print(data.decode('shift_jis'))           # 繒繒 s 繒繒繒繒繒 ← 文字化け。エラーにならない
>>> print(repr(data))                         # b'\xe3\x81\x93\xe3\x82\x93\xe3\x81\xab...' ← 先頭が \xe3
```

```
>>> data = 'こんにちは'.encode('shift_jis')
>>> print(data.decode('utf-8', errors='replace')) # 繒繒繒繒繒繒繒 ← 読めない部分が 繒 になる
>>> print(repr(data))                           # b'\x82\xb1\x82\xf1\x82\xc9...' ← 先頭が \x82
```

- 化けた文字列を見ても、元のバイト列は分からない。 `repr(data)` で、 `decode()` する前のバイト列を見る
- 日本語のバイト列の見分け方
 - UTF-8 は `\xe3` など始まり、3 バイトで 1 文字
 - Shift_JIS は `\x82` など始まり、2 バイトで 1 文字
- バイト列の様子から、相手がどの文字コードで送ったかを推測し、 `decode()` の文字コードを合わせる

2. with 文

with 文 (1) — 後片付けを書き忘れない仕組み

ソケットやファイルは、使い終わったら閉じる必要がある。

```
# with を使わないと
s = socket.socket(...)
s.connect((HOST, PORT))
s.sendall(b'Hello')
data = s.recv(1024)
s.close()      # ← 書き忘れる
               #  途中で例外が起きると実行されない
```

```
# with なら
with socket.socket(...) as s:
    s.connect((HOST, PORT))
    s.sendall(b'Hello')
    data = s.recv(1024)
# ← ここを抜けた時点で必ず close() される
```

- 閉じ忘れたソケットは、プログラムが動いている間ポートを使い続ける
- 前のサーバが動いたままだと、次の起動が `OSError: Address already in use` で失敗する

with 文 (2) — 書き方と動き

```
with 式 as 名前:
    ブロック          # インデントした部分
```

タイミング	起きること
① with に入るとき	式 を評価し、その結果を 名前 に代入する (socket.socket(...) ならソケットができる)
② ブロックの中	普通のコードを実行する。 名前 を使って送受信する
③ ブロックを抜けるとき	自動で 名前.close() が実行される。正常終了でも、 break でも、例外でも同じ

- ファイルでも同じように使える (with open(file) as f:)。第 5 回で使う
- echoServer.py では with が 2 回出てくる。 socket 用 (s) と、接続ごとの conn 用

C で fopen() したら必ず fclose() を書くのと同じ。Python では、with 文がその後片付けを自動で行う。

3. echoClient.py を読む

echoClient.py

```
import socket

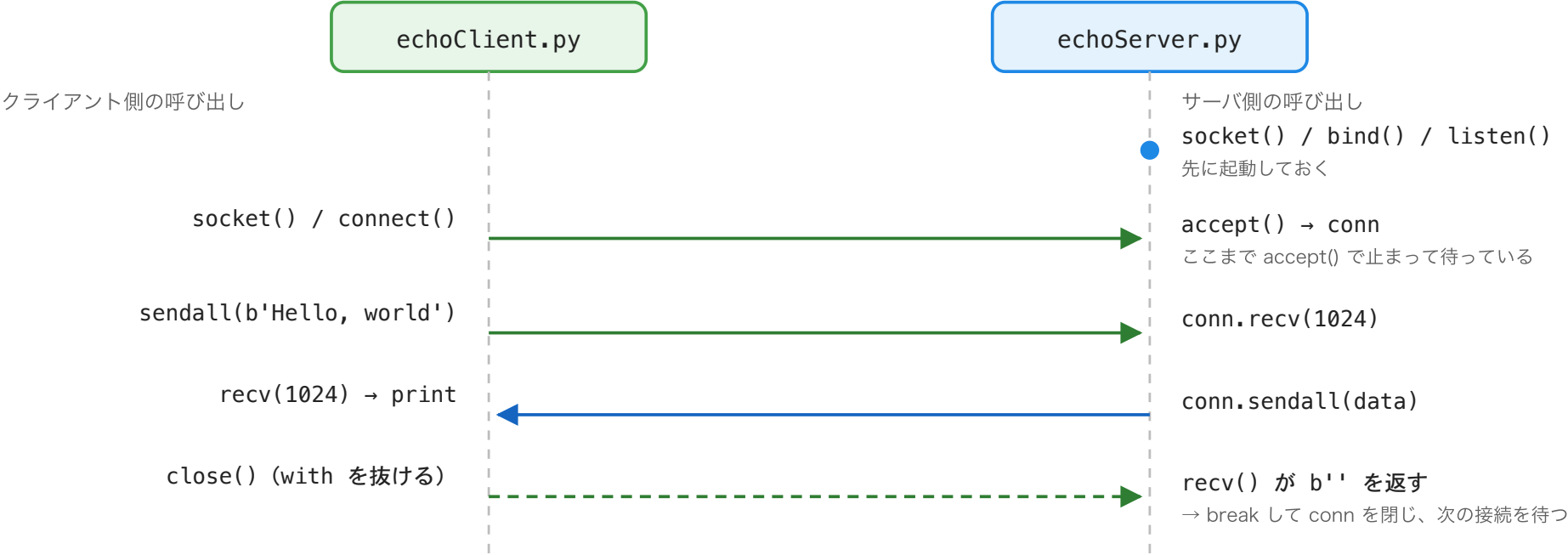
HOST = 'localhost'    # 接続先のホスト名
PORT = 50007          # 相手のポート番号

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    send = b'Hello, world'
    s.sendall(send)
    print('Send', repr(send))
    data = s.recv(1024)
    print('Received', repr(data))
```

- `AF_INET` = IPv4、`SOCK_STREAM` = TCP。どちらも `socket` モジュールが持つ定数
- `connect()` の引数はタプル 1 個 (`str`, `int`)。 `connect(HOST, PORT)` ではない
- `recv(1024)` の 1024 は、受け取る最大のバイト数。毎回ちょうど 1024 バイト届くわけではない

4. echoServer.py を読む

echoServer.py の全体の処理



矢印 1 本ごとに、両側で対になる呼び出しがある。片方だけ書くと、もう片方はそこで待ち続ける。

全体の流れ — ループが 2 つある

起動: 待受けの準備をする

socket() → bind() → listen()

外側のループ (接続ごと)

| accept() 接続が来るまで待つ
| → 相手専用の conn ができる

| 内側のループ (データごと)

| | recv() 相手の送ったデータを受け取る
| | sendall() そのまま送り返す
| | recv() が b'' なら、抜ける
| conn を閉じて、次の accept() へ戻る

- 外側のループは接続の単位。1 つの接続が終わったら、次の接続を受け付ける
 - このループを抜けると、サーバが終了する
- 内側のループはデータの単位。1 つの接続の中で、何度でも送受信できる
 - recv() が b'' (空のバイト列) を返したら、相手が接続を閉じたことを表す
 - これを書かないと無限ループになる

echoServer.py (コード全体)

```
import socket

HOST = ''          # 空文字列 = この PC のどのアドレス宛でも受ける
PORT = 50007

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:    # ① 待受け用のソケット s
    s.bind((HOST, PORT))    # ② ソケットをこのポートに結びつける
    s.listen(1)             # 待受けを開始する
    while True:             # ③ 接続を受け続ける (外側のループ)
        conn, addr = s.accept() # 接続が来るまでここで止まる
        with conn:          # ④ この相手専用の conn
            print('Connected by', addr)
            while True:      # ⑤ この接続の中でデータを受け続ける (内側のループ)
                data = conn.recv(1024)
                if not data:  # 相手が閉じた
                    break
            conn.sendall(data)
```

- ・ クライアントには無く、サーバだけが呼ぶのは `bind()` / `listen()` / `accept()` の 3 つ
- ・ 以降、①～⑤ の順に読んでいく

① ② 待受け用のソケットを作る

```
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:    # ① ソケット s を作る
    s.bind((HOST, PORT))    # ② ポート 50007 に結びつける
    s.listen(1)    # 待受けを開始する
```

- `socket.socket(...)` — クライアントと同じ。IPv4 の TCP ソケット `s` を作る
- `bind((HOST, PORT))` — `s` を、この PC のポート 50007 に結びつける
 - `HOST` が空文字列 `''` なら、この PC のどのアドレス宛の接続も受ける
 - 引数は `connect()` と同じくタプル 1 個
- `listen(1)` — `s` を「接続を待つ状態」にする。1 は、受付前に待たせておける接続数の目安
- この `s` は、接続を受け付けるためだけのソケット。データの送受信には使わない

③ accept() — ソケットが 2 つある理由 (1/5)



手順 1 : サーバを起動する。ソケットは待受け用の `s` が 1 つだけ

ポート 50007 を押さえて、`accept()` の行で止まって待つ。クライアントが来るまで、ここから進まない。

```
s.bind() / s.listen() / s.accept()
```

③ accept() — ソケットが 2 つある理由 (2/5)



手順 2：クライアントが `connect()` する。接続要求を受けるのは `s`

`s` は「受付」。ここでは、まだ送受信の相手になるソケットが無い。

```
client: s.connect((HOST, PORT))
```

③ accept() — ソケットが 2 つある理由 (3/5)



手順 3 : `accept()` が、この相手専用のソケット `conn` を新しく作って返す

`accept()` の行で止まっていたサーバが動き出す。s は消えずに残り、`conn` が増える。

```
conn, addr = s.accept()
```

③ accept() — ソケットが 2 つある理由 (4/5)



手順 4：送受信は、クライアントと conn の間だけで行う

s に `recv()` や `sendall()` をしても相手には届かない。s の役目は、接続を受け付けて `conn` を作るところまで。

```
conn.recv() / conn.sendall()
```

③ accept() — ソケットが 2 つある理由 (5/5)



手順 5 : 相手が閉じたら `conn` を閉じ、`s` で次の `accept()` を待つ

接続ごとに `conn` が作られ、閉じられる。`s` は、サーバを終了するまで 1 つだけ残り続ける。

`with conn:` を抜けて、`accept()` へ戻る

③ accept() — ソケットが 2 つある理由（まとめ）

ソケット	作る場所	役割	数
s	サーバ起動時（ <code>socket.socket()</code> ）	接続を受け付ける。データは送受信しない	1 つだけ
conn	接続ごと（ <code>s.accept()</code> の返り値）	その相手と送受信する	接続のたびに新しく作る

- 受付のソケットが相手専用になってしまうと、次の接続を受け付けられない
- そのため、受付（`s`）と、相手ごとの通信（`conn`）に分ける
- `addr` は、相手のアドレスとポート番号のタプル（例: `('127.0.0.1', 51000)`）

④ ⑤ conn でデータを送受信する

```
with conn:                # ④ conn は、with を抜けると自動で閉じる
    print('Connected by', addr)
    while True:            # ⑤ 内側のループ
        data = conn.recv(1024) # 相手のデータを最大 1024 バイト受け取る
        if not data:          # b'' = 相手が接続を閉じた
            break              # 内側のループを抜ける
        conn.sendall(data)    # 受け取ったバイト列を、そのまま送り返す
```

- `recv()` と `sendall()` は、`s` ではなく `conn` に対して行う
- `data` は `bytes` 。 `encode()` / `decode()` を使わず、そのまま返せる (echo は中身を解釈しない)
- `break` で内側のループを抜けると、`with conn:` の終わりで `conn` が閉じる。外側のループが、次の `accept()` へ戻る
- TCP はストリームなので、送信 1 回が受信 1 回に対応するとは限らない
 - `sendall(b'Hello, world')` が `b'Hello, '` と `b'world'` に分かれて届くこともある
 - どこまでが 1 つのメッセージかは、アプリケーション層のプロトコルが決める。第 5 回の HTTP では、空行までがその区切りになる

今日やること・次回

5 講時（1-608 で実習）

1. `echoServer.py` を動かしたまま、`echoClient.py` の送信内容を日本語に変えてみる
 - `b'こんにちは'` と書けないのはなぜか。 `'こんにちは'.encode()` なら動くのはなぜか
2. `print(repr(data))` を追加して、実際に何が届いているかを確認める
3. aiJudge の **ex4** に取り組む
4. paiza ラーニング「新・Python入門編」21～22（2 レッスン）の認定証を、まとめて 1 回で aiJudge へ提出

次回（10/16）：HTTP プロトコルと `httpServer.py`

- HTTP リクエスト／レスポンスの形、URL の読み方、ファイル処理と `try` ～ `except`

質問は Teams 「★ 質問用チャンネル」 か、教室の TA へ。